

Lab 3 Report:

MNIST Classification with FCN

Name:

```
In [3]: # Import necessary packages
from torch.utils.data import TensorDataset, DataLoader

%matplotlib inline

import matplotlib.pyplot as plt

import torch
import torchvision
import numpy as np
import random
from sklearn.preprocessing import StandardScaler
```

```
In [4]: # Arthur: I add this because seed could make every output the same when setting
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

set_seed(42)
```

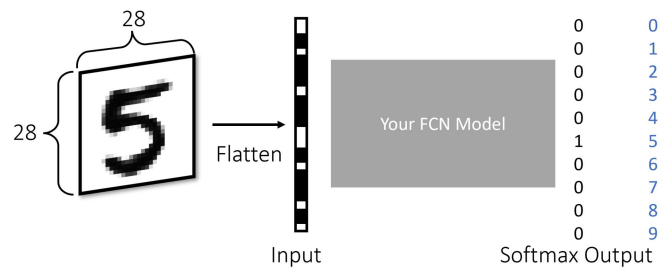
```
In [5]: from IPython.display import Image # For displaying images in colab jupyter cell
```

```
In [6]: Image('lab3_exercise.PNG', width = 1000)
```

Out[6]:



MNIST Classification with FCN



In this exercise, you will classify handwritten digits (28 x 28) using your own Fully Connected Network Architecture.

Prior to training your neural net, 1) Flatten each digit into 1D array of size 784, 2) Normalize the dataset using standard scaler and 3) Split the dataset into train/validation/test.

Design your own neural net architecture with your choice of hidden layers, activation functions, optimization method etc.

Your goal is to achieve a testing accuracy of >90%, with no restrictions on epochs.

Demonstrate the performance of your model via plotting the training loss, validation accuracy and printing out the testing accuracy.

Plot the testing samples where your model failed to classify correctly and print your model's best guess for each of them

44

Prepare Data

```
In [8]: # Load MNIST Dataset in Numpy

# 1000 training samples where each sample feature is a greyscale image with shape (28, 28)
# 1000 training targets where each target is an integer indicating the true digit

mnist_train_features = np.load('mnist_train_features.npy')
mnist_train_targets = np.load('mnist_train_targets.npy')

# 100 testing samples + targets
mnist_test_features = np.load('mnist_test_features.npy')
mnist_test_targets = np.load('mnist_test_targets.npy')

# Print the dimensions of training sample features/targets
print(mnist_train_features.shape, mnist_train_targets.shape)
# Print the dimensions of testing sample features/targets
print(mnist_test_features.shape, mnist_test_targets.shape)

print(mnist_train_features, mnist_train_targets)
```

```

(1000, 28, 28) (1000,)
(100, 28, 28) (100,)
[[[0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  ...
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]]]

[[[0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  ...
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]]]

[[[0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  ...
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]]]

...

[[[0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  ...
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]]]

[[[0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  ...
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]]]

[[[0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  ...
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]
  [0 0 0 ... 0 0 0]]] [5 0 4 1 9 2 1 3 1 4 3 5 3 6 1 7 2 8 6 9 4 0 9 1 1 2 4 3 2
7 3 8 6 9 0 5 6
0 7 6 1 8 7 9 3 9 8 5 9 3 3 0 7 4 9 8 0 9 4 1 4 4 6 0 4 5 6 1 0 0 1 7 1 6
3 0 2 1 1 7 9 0 2 6 7 8 3 9 0 4 6 7 4 6 8 0 7 8 3 1 5 7 1 7 1 1 6 3 0 2 9
3 1 1 0 4 9 2 0 0 2 0 2 7 1 8 6 4 1 6 3 4 5 9 1 3 3 8 5 4 7 7 4 2 8 5 8 6
7 3 4 6 1 9 9 6 0 3 7 2 8 2 9 4 4 6 4 9 7 0 9 2 9 5 1 5 9 1 2 3 2 3 5 9 1
7 6 2 8 2 2 5 0 7 4 9 7 8 3 2 1 1 8 3 6 1 0 3 1 0 0 1 7 2 7 3 0 4 6 5 2 6
4 7 1 8 9 9 3 0 7 1 0 2 0 3 5 4 6 5 8 6 3 7 5 8 0 9 1 0 3 1 2 2 3 3 6 4 7
5 0 6 2 7 9 8 5 9 2 1 1 4 4 5 6 4 1 2 5 3 9 3 9 0 5 9 6 5 7 4 1 3 4 0 4 8
0 4 3 6 8 7 6 0 9 7 5 7 2 1 1 6 8 9 4 1 5 2 2 9 0 3 9 6 7 2 0 3 5 4 3 6 5

```

```

8 9 5 4 7 4 2 7 3 4 8 9 1 9 2 8 7 9 1 8 7 4 1 3 1 1 0 2 3 9 4 9 2 1 6 8 4
7 7 4 4 9 2 5 7 2 4 4 2 1 9 7 2 8 7 6 9 2 2 3 8 1 6 5 1 1 0 2 6 4 5 8 3 1
5 1 9 2 7 4 4 4 8 1 5 8 9 5 6 7 9 9 3 7 0 9 0 6 6 2 3 9 0 7 5 4 8 0 9 4 1
2 8 7 1 2 6 1 0 3 0 1 1 8 2 0 3 9 4 0 5 0 6 1 7 7 8 1 9 2 0 5 1 2 2 7 3 5
4 9 7 1 8 3 9 6 0 3 1 1 2 6 3 5 7 6 8 3 9 5 8 5 7 6 1 1 3 1 7 5 5 5 2 5 8
7 0 9 7 7 5 0 9 0 0 8 9 2 4 8 1 6 1 6 5 1 8 3 4 0 5 5 8 3 6 2 3 9 2 1 1 5
2 1 3 2 8 7 3 7 2 4 6 9 7 2 4 2 8 1 1 3 8 4 0 6 5 9 3 0 9 2 4 7 1 2 9 4 2
6 1 8 9 0 6 6 7 9 9 8 0 1 4 4 6 7 1 5 7 0 3 5 8 4 7 1 2 5 9 5 6 7 5 9 8 8
3 6 9 7 0 7 5 7 1 1 0 7 9 2 3 7 3 2 4 1 6 2 7 5 5 7 4 0 2 6 3 6 4 0 4 2 6
0 0 0 0 3 1 6 2 2 3 1 4 1 5 4 6 4 7 2 8 7 9 2 0 5 1 4 2 8 3 2 4 1 5 4 6 0
7 9 8 4 9 8 0 1 1 0 2 2 3 2 4 4 5 8 6 5 7 7 8 8 9 7 4 7 3 2 0 8 6 8 6 1 6
8 9 4 0 9 0 4 1 5 4 7 5 3 7 4 9 8 5 8 6 3 8 6 9 9 1 8 3 5 8 6 5 9 7 2 5 0
8 5 1 1 0 9 1 8 6 7 0 9 3 0 8 8 9 6 7 8 4 7 5 9 2 6 7 4 5 9 2 3 1 6 3 9 2
2 5 6 8 0 7 7 1 9 8 7 0 9 9 4 6 2 8 5 1 4 1 5 5 1 7 3 6 4 3 2 5 6 4 4 0 4
4 6 7 2 4 3 3 8 0 0 3 2 2 9 8 2 3 7 0 1 1 0 2 3 3 8 4 3 5 7 6 4 7 7 8 5 9
7 0 3 1 6 2 4 3 4 4 7 5 9 6 9 0 7 1 4 2 7 3 6 7 5 8 4 5 5 2 7 1 1 5 6 8 5
8 4 0 7 9 9 2 9 7 7 8 7 4 2 6 9 1 7 0 6 4 2 5 7 0 7 1 0 3 7 6 5 0 6 1 5 1
7 8 5 0 3 4 7 7 5 7 8 6 9 3 8 6 1 0 9 7 1 3 0 5 6 4 4 2 4 4 3 1 7 7 6 0 3
6]

```

In [9]: *# Let's visualize some training samples*

```

plt.figure(figsize = (10, 10))

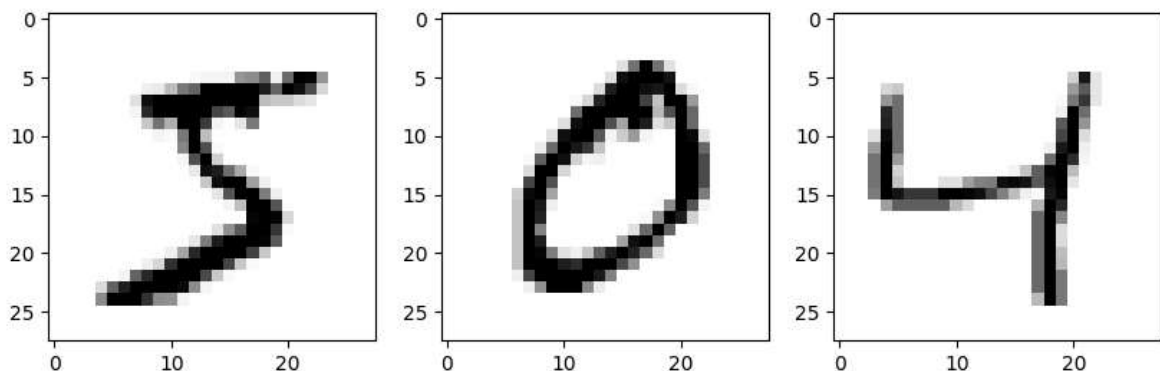
plt.subplot(1,3,1)
plt.imshow(mnist_train_features[0], cmap = 'Greys')

plt.subplot(1,3,2)
plt.imshow(mnist_train_features[1], cmap = 'Greys')

plt.subplot(1,3,3)
plt.imshow(mnist_train_features[2], cmap = 'Greys')

```

Out[9]: <matplotlib.image.AxesImage at 0x149dfcb32b0>



```

In [10]: # Reshape features via flattening the images
# This refers to reshape each sample from a 2d array to a 1d array.
# hint: np.reshape() function could be useful here

# Arthur: I flatten them from 3D to 2D, which means 1000, 28, 28 to 1000, 784
print(mnist_train_features.shape)
print(mnist_test_features.shape)

# m_t_f = np.empty((1000,784))
# count = 0
# for ele in mnist_train_features:
#     m_t_f[:, count]= ele.reshape(-1)
#     count = count + 1

```

```
# Arthur: Add training features Loss List
m_train_f = []
for ele in mnist_train_features:
    m_train_f.append(ele.reshape(-1))
m_train_f = np.array(m_train_f)

# Arthur: Add testing features Loss List
m_test_f = []
for ele in mnist_test_features:
    m_test_f.append(ele.reshape(-1))
m_test_f = np.array(m_test_f)

mnist_train_features = m_train_f#YOUR CODE HERE
mnist_test_features = m_test_f#YOUR CODE HERE

print(mnist_train_features.shape, mnist_test_features.shape)
```

```
(1000, 28, 28)
(100, 28, 28)
(1000, 784) (100, 784)
```

```
In [11]: # Scale the dataset according to standard scaling
# Arthur: Yes I am scaling these values for standarlizing these vectors for bett
scaler = StandardScaler()
mnist_train_features = scaler.fit_transform(mnist_train_features)# YOUR CODE HER
mnist_test_features = scaler.fit_transform(mnist_test_features)# YOUR CODE HERE
print(mnist_train_features.shape, mnist_test_features.shape)
```

```
(1000, 784) (100, 784)
```

```
In [12]: # Split training dataset into Train (90%), Validation (10%)
# Arthur: I split these features by the ratio 9:1. Then using print shape for te
length = int(len(mnist_train_features)*0.1)

mnist_validation_features = mnist_train_features[:length]# YOUR CODE HERE
mnist_validation_targets = mnist_train_targets[:length]# YOUR CODE HERE

mnist_train_features = mnist_train_features[length:]# YOUR CODE HERE
mnist_train_targets = mnist_train_targets[length:]# YOUR CODE HERE
print(mnist_validation_features.shape, mnist_validation_targets.shape, mnist_tra
```

```
(100, 784) (100,) (900, 784) (900,)
```

Define Model

```
In [14]: class mnistClassification(torch.nn.Module):
# Arthur: The class I basically get them from the Lab 3 example. We have fou
def __init__(self, input_dim, output_dim, hidden1_dim, hidden2_dim): # Feel

    super(mnistClassification, self).__init__()

    self.layer1 = torch.nn.Linear(input_dim, hidden1_dim) # Connections f
    self.layer2 = torch.nn.Linear(hidden1_dim, hidden2_dim) # Connections f
    self.layer3 = torch.nn.Linear(hidden2_dim, output_dim) # Connections f
    # YOUR CODE HERE

    #(@•ᵕ•)ᵕ Nothing I just think this emoji is cute.
    # Arthur: Forward, also get this from the example code. Forward is for conne
    # numbers, and having the new results forward to other neural layers.
```

```
def forward(self, x):

    out1 = torch.nn.functional.relu(self.layer1(x))          # Apply relu ac
    out2 = torch.nn.functional.relu(self.layer2(out1))        # Apply relu ac
    output = self.layer3(out2)
    # YOUR CODE HERE

    return output
```

Define Hyperparameters

```
In [53]: # Initialize our neural network model with input and output dimensions

# Arthur: I wanna write a for loop next time for getting the maximum accuracy!!

model = mnistClassification(input_dim = 784, output_dim = 10, hidden1_dim = 1000

# Define the Learning rate and epoch
learning_rate = 0.001# YOUR CODE HERE
epochs = 128# YOUR CODE HERE
batchsize = 64# YOUR CODE HERE (If you are using mini-batch gradient)

# Define Loss function and optimizer
loss_func = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)# YOUR CODE

# Arthur: Yes I am using GPU instead of CPU. Be aware of this transformation! Be
# with the same code. Check the datatype next time.

# Run this Line if you have PyTorch GPU version
if torch.cuda.is_available():
    model.cuda()

model
```

```
Out[53]: mnistClassification(
  (layer1): Linear(in_features=784, out_features=1000, bias=True)
  (layer2): Linear(in_features=1000, out_features=150, bias=True)
  (layer3): Linear(in_features=150, out_features=10, bias=True)
)
```

Identify Tracked Values

```
In [57]: # Placeholders for training loss and validation accuracy during training
# Training Loss should be tracked for each iteration (1 iteration -> single forw
# Validation accuracy should be evaluated every 'Epoch' (1 epoch -> full trainin
# If using batch gradient, 1 iteration = 1 epoch

# Arthur: Yes this is train_loss_list and validation_accuracy_list with the Leng

train_loss_list = np.zeros((epochs,))# YOUR CODE HERE
validation_accuracy_list = np.zeros((epochs,))# YOUR CODE HERE

max_accuracy = 0
best_epoch = 0
```

Train Model

```

In [59]: import tqdm

# Convert the training, validation, testing dataset (NumPy arrays) into torch tensors
#

# YOUR CODE HERE
train_inputs = torch.from_numpy(mnist_train_features).float().cuda()
train_targets = torch.from_numpy(mnist_train_targets).long().cuda() # Convert to long

validation_inputs = torch.from_numpy(mnist_validation_features).float().cuda()
validation_targets = torch.from_numpy(mnist_validation_targets).long().cuda()

testing_inputs = torch.from_numpy(mnist_test_features).float().cuda()
testing_targets = torch.from_numpy(mnist_test_targets).long().cuda()
# Training Loop -----

for epoch in tqdm.trange(epochs):

    # YOUR CODE HERE

    # Compute Validation Accuracy -----

    # YOUR CODE HERE

    optimizer.zero_grad() # Arthur: Calculating Gradient.

    train_outputs = model(train_inputs) # Arthur: Training the model, feeding it
    loss = loss_func(train_outputs, train_targets) # Arthur: Calculating loss function
    train_loss_list[epoch] = loss.item() # Arthur: Calculating training loss.

    loss.backward() # Arthur: Backward gradient.

    optimizer.step() # Arthur: Renewing the model.

    # Compute Validation Accuracy -----

    with torch.no_grad(): # Telling PyTorch we aren't passing inputs to network

        # Pass the validation feature data (30 samples) to the network
        validation_outputs = model(validation_inputs)

        # validation_outputs = tensor of shape (30, 3), i.e., 30 predictions and
        # These are classification probabilities for each flower type: [p(setosa), p(versicolour), p(virginica)]
        # torch.argmax(validation_outputs, dim=1) finds the index with the maximum probability
        # i.e. for each sample, it finds the column index with the highest probability
        # == validation_targets compares these indices with groundtruth validation targets
        # For each sample, it returns True if the index matches the target, False otherwise
        # .type(torch.FloatTensor) converts True = 1, False = 0
        # Finally .mean() gives us (Total count of 1) / (Length of the 1D tensor)

        correct = (torch.argmax(validation_outputs, dim=1) ==
                    validation_targets).type(torch.FloatTensor)

        validation_accuracy_list[epoch] = correct.mean()

```

```
100%|██████████| 128/128 [00:01<00:00, 115.80it/s]
```

Visualize and Evaluate Model

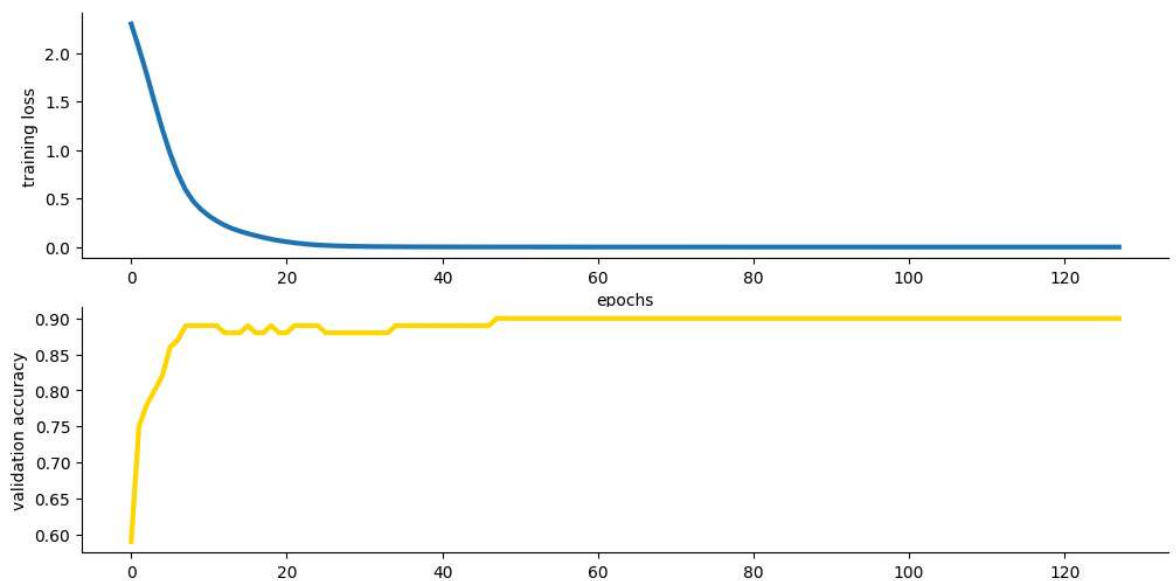
```
In [22]: # Import seaborn for prettier plots
import seaborn as sns
```

```
In [61]: # Visualize training loss

plt.figure(figsize = (12, 6))

# Visualize training loss with respect to iterations (1 iteration -> single batch)
plt.subplot(2, 1, 1)
plt.plot(train_loss_list, linewidth = 3)
plt.ylabel("training loss")
plt.xlabel("epochs")
sns.despine()

# Visualize validation accuracy with respect to epochs
plt.subplot(2, 1, 2)
plt.plot(validation_accuracy_list, linewidth = 3, color = 'gold')
plt.ylabel("validation accuracy")
sns.despine()
```



```
In [63]: # Compute the testing accuracy
# print(train_outputs)
# print(type(mnist_train_targets))

# Arthur: Calculating the model's final accuracy.
# Arthur: Comparing the model accuracy with validation accuracy for knowing whet

train_Data = torch.argmax(train_outputs, dim=1)
# print(train_Data)

train_Result = torch.from_numpy(mnist_train_targets).cuda()
train_match_ones = train_Result == train_Data
# print(match_ones)

print(validation accuracy list[-1])
```

```

test_Data = torch.argmax(model(testing_inputs), dim=1)
test_match_ones = testing_targets == test_Data
test_correct_accuracy = (test_Data == testing_targets).type(torch.FloatTensor)
print(test_correct_accuracy.mean())
print(testing_targets.shape)
# YOUR CODE HERE

```

0.8999999761581421

tensor(0.9000)

torch.Size([100])

```

In [67]: # Plot 5 incorrectly classified testing samples and print the model predictions
# You can use np.reshape() to convert flattened 1D array back to 2D array

# Arthur: Some of the plots could be recognizing by eyes, but some of them cannot
incorrect_samples_set = []
for i in range(0, len(testing_targets)):
    if testing_targets[i] != test_Data[i]:
        incorrect_samples_set.append(i)
print(incorrect_samples_set)
# YOUR CODE HERE

for i in range(0, 5):
    plt.subplot(1,5, i+1)
    number = incorrect_samples_set[i]
    plt.imshow(testing_inputs[number].cpu().numpy().reshape(28, 28), cmap = 'Gre
    print(test_Data[number].item())

```

[33, 42, 44, 61, 63, 65, 66, 73, 80, 97]

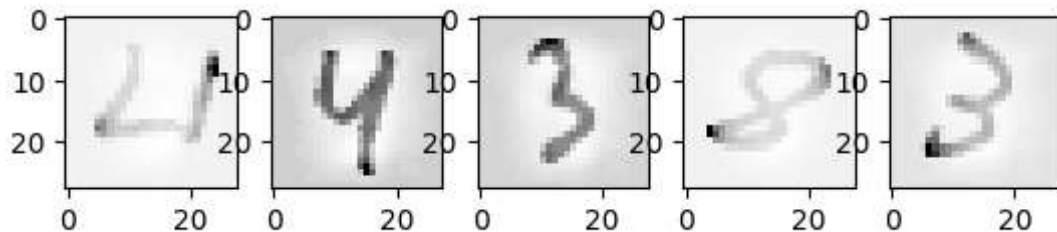
0

9

5

2

2



In []: